

CS3106L - Luit Lab 1 Report

Name: Shreyansh Kumar
Roll number: 240101094
Laboratory section: CSE

Q1 - Reading map

File	Role / Purpose
kernel/syscall.tbl	The source file acting as a single source of truth that defines system calls. It maps syscall numbers to their respective names and kernel handler functions. It is manually edited when a new syscall is introduced.
kernel/syscalltab.h	A generated kernel header file that defines the function pointer array (<code>syscalls[]</code>). It maps the syscall number directly to its actual C handler function (<code>sys_sleep</code> , etc.) used by the kernel dispatcher.
user/usys.S	A generated user-space assembly file containing syscall wrapper functions (stubs). These wrappers load the correct syscall number into a register and execute the <code>ecall</code> instruction to trap into the kernel safely.
kernel/syscallnums.h	A generated header file that defines the <code>SYS_<name></code> integer constants (e.g., <code>SYS_sleep</code>) which serve as unique identifiers for system calls shared across both kernel and user space.

Q2 - Argument validation and syscall semantics

Using `atoi()` alone is insufficient for robust argument validation because `atoi()` silently handles invalid formatting and fails to detect integer overflows. For example, it will parse the string "12x" as simply 12, discarding the invalid suffix without throwing any error. It also silently wraps around on extremely large numbers, and doesn't explicitly reject empty strings or completely non-numeric inputs. This leads to unpredictable system behaviour, as invalid inputs are masqueraded as perfectly valid integers, creating bugs that are hard to trace.

For the `sleep` command, the argument must strictly be a valid non-negative integer representing the exact number of ticks. Therefore, explicit validation (using methods like `strtol` or manual digit-by-digit checking) is required to ensure that the input contains only valid digits and that its numerical value sits safely within the valid integer bounds before making the system call.

Furthermore, in the baseline kernel, if a negative value is passed and reaches `sys_sleep()`, it is cast to an unsigned integer. Due to the two's complement binary representation, a negative number like -1 mathematically becomes an extremely large unsigned positive value. The kernel would then attempt to sleep until the system tick counter reaches this massive value. This causes the process to effectively hang forever, wasting CPU cycles and permanently occupying one of the limited process slots within the operating system's process table.

Q3 - Pipe ownership

When the `fork()` system call is invoked, the child process inherits a complete copy of the parent's file descriptor table. Thus, immediately after `fork()`, both the parent and child processes have open descriptors for both ends of the two pipes.

Process	Descriptor	Direction (Role)
Parent	p2c[0]	Read end of parent-to-child pipe (Unused)
Parent	p2c[1]	Write end of parent-to-child pipe (Used)
Parent	c2p[0]	Read end of child-to-parent pipe (Used)
Parent	c2p[1]	Write end of child-to-parent pipe (Unused)
Child	p2c[0]	Read end of parent-to-child pipe (Used)
Child	p2c[1]	Write end of parent-to-child pipe (Unused)
Child	c2p[0]	Read end of child-to-parent pipe (Unused)
Child	c2p[1]	Write end of child-to-parent pipe (Used)

After both processes execute the code to close their unused pipe ends, the descriptor ownership optimally becomes:

Process	Descriptor	Operation
Parent	p2c[1]	Write token to child
Parent	c2p[0]	Read reply from child
Child	p2c[0]	Read token from parent
Child	c2p[1]	Write reply to parent

The underlying file descriptor structure tracks a system-wide reference count of open ends. A pipe reaches End-Of-File (EOF) on the read side (causing `read()` to return 0) only when **all** file descriptors referring to the write end of that pipe are completely closed across the entire system. If an extra write end remains open (for example, if the child forgets to close `p2c[1]`), the OS assumes that a writer might still produce data in the future. As a result, the reading process will continue to hang indefinitely in a blocked state, waiting for more data, rather than correctly terminating when the intended writer exits.

Q4 - Synchronisation without delays

The deterministic ordering of the console output is strictly guaranteed by the blocking nature of the `read()` system calls interacting over the two pipes. The parent process writes a byte to the parent-to-child pipe. The child process performs a `read()` on this pipe. Since `read()` is a blocking operation, the child is placed into a sleeping state until the parent successfully sends the data.

After receiving the byte, the child successfully prints the “received ping” message to the console, and then proceeds to write a response byte back into the child-to-parent pipe. The parent process, meanwhile, is performing a blocking `read()` on this second pipe, meaning it is effectively paused and cannot print the “received pong” message until the child has fully completed its printing and sent the response byte.

Therefore, the strict happens-before relationships established are:

Parent write ping → Child read ping

Child print ping → Child write pong

Child write pong → Parent read pong

Parent read pong → Parent print pong

These absolute dependencies mathematically force the child's output line to always appear before the parent's output line, without relying on any arbitrary or artificial delays.

Adding a `sleep()` call would simply inject an arbitrary timing delay and hide a fundamental design error. It would not establish a correct data-flow synchronization relationship. While it might appear to fix the ordering visually during a single test run, it relies on timing assumptions that will easily break under heavy system loads or different scheduler conditions. Using `sleep()` merely sweeps the problem under the rug, whereas pipes provide true structural synchronization because they forcibly block processes until the necessary communication event physically occurs.

Q5 - Constructing the process tree

Merely sorting the snapshot records returned by `procstat()` by their PID values is insufficient for constructing a hierarchical process tree. A PID only sequentially identifies a process; while newer processes tend to have larger PIDs, this linearity does not group parents with their children or represent ancestry. The actual hierarchy must be mathematically reconstructed by traversing the explicit `ppid` (parent PID) fields from each process record.

In my representation, parent/child relationships are established by performing a Depth-First Search (DFS) directly on the snapshot array. Starting from a specified `root_pid`, we search the array for all processes where their `ppid` precisely matches the `root_pid`. The tree is generated by recursively finding and printing these discovered child processes.

The initial array is still sorted by PID beforehand to ensure deterministic and neat ordering between siblings. This guarantees that processes sharing the exact same parent are printed sequentially in increasing PID order, even while the overarching tree structure is still strictly dictated by the parent-child PID links.

Crucially, a separate `visited[]` array is maintained during this recursive traversal. A process record's specific index is marked as visited before recursively processing its own children. This entirely prevents duplicate printing and cleanly aborts infinite recursion if a changing snapshot happens to create a malformed topological cycle.

The following is an actual snapshot output obtained from running my `pstree` implementation while multiple `vdemo` processes were actively forking in the background:

```
init(1) [sleep]
  sh(2) [sleep]
    pstree(10) [run]
      vdemo(5) [sleep]
        vdemo(6) [sleep]
        vdemo(7) [sleep]
        vdemo(8) [sleep]
        vdemo(9) [sleep]
```

This perfectly demonstrates the multi-generational tree construction: `sh` and `vdemo(5)` are direct children of `init`, and the cascade of subsequent `vdemo` processes correctly branch off as descendants.

Q6 - Snapshot consistency

The `procstat()` system call reads process records safely, but the underlying live process table is continuously changing while the iterative scan is taking place. Because the kernel doesn't freeze the entire system during this scan, the returned snapshot may not perfectly represent the exact state of

the process table at one single atomic instant, leading to a classic Time-Of-Check to Time-Of-Use (TOCTOU) race condition.

A concrete sequence of events demonstrating this:

1. `procstat()` starts scanning and correctly records Process A as having parent B.
2. Right after A is successfully scanned, but just before the scan reaches B's slot, parent B exits.
3. B's process slot is immediately reused by a brand new process C.
4. The scan reaches that slot and unknowingly records C instead of B.

As a result, the returned user-space snapshot shows A having parent B, but process B literally does not exist in the snapshot table. Alternatively, rapid PID reuse during the scan could result in A being recorded as the parent of B, while B is later recorded as the parent of A, forming a physically impossible cycle in the graph.

My program remains completely safe against these inconsistencies because it never blindly trusts the links. It independently verifies the existence of parents and children using a helper function `find_process_index()` and simply ignores missing links, treating those orphaned processes as isolated nodes. To safeguard against cycles, it explicitly marks each array index in a boolean `visited[]` array before recursing. If it encounters an already-visited index during the descent, it immediately detects the cycle, prints a cycle warning, and returns, completely preventing a fatal infinite loop.

Q7 - Generated ABI and fault injection

The file `kernel/syscall.tbl` acts as the single source of truth (like an IDL) defining the system call interface. The dynamically generated files, such as the user-space assembly stubs (`user/usys.S`) and kernel headers (`kernel/syscalltab.h` and `kernel/syscallnums.h`), are automatically synthesized from this singular table. Programmatically generating all Application Binary Interface (ABI) files from one source prevents catastrophic mismatches where a developer might add or renumber a system call in the kernel but forget to update the corresponding assembly stub in user-space, causing the user program to accidentally trigger the wrong kernel handler.

For fault testing, I deliberately broke the generated ABI by modifying the user-space assembly file `user/usys.S`. Specifically, I changed the syscall number of the `exec` stub to an incorrect value.

When I subsequently ran the checker script using:

```
python3 tools/syscallmap.py
```

The script successfully detected the deliberate inconsistency and threw a hard error:

```
ERROR: missing or mismatched SYSCALL exec, 7 stub in usys.S
```

This proved the ABI was invalid and the checker was working correctly.

The error was cleanly repaired by regenerating the assembly files using the script:

```
python3 tools/gensyscalls.py
```

This command read from `syscall.tbl` and completely overwrote my modified `usys.S`, restoring the true `exec` syscall number. Running the checker again then correctly reported `ABI valid`, confirming that single-source generation gracefully repairs manual desynchronization.

Q8 - System-call path

The execution of the `sleep` system call originates in user space when the C program invokes `sleep(5)`. This directly jumps to the generated user stub inside `usys.S`, which prepares the CPU for the context switch into the kernel by using the RISC-V `ecall` instruction.

My disassembled user stub for `sleep` confirms this sequence:

```
00000000000017f2 <sleep>:
    17f2:      48b5             li      a7,13
    17f4:      00000073         ecall
    17f8:      8082             ret
```

The value loaded into register `a7` is 13, which correctly corresponds to the `sleep` syscall number defined in our ABI table. The argument (5) is held in the `a0` register as per RISC-V calling conventions.

During the hardware transition into the kernel, the CPU traps to `usertrap()`. My compact GDB transcript captured the exact state at this boundary:

```
(gdb) p $p->pid
$1 = 3
(gdb) p $p->name
$2 = "sleep\000\000\000\000\000\000\000\000\000\000"
(gdb) p $p->tf->a7
$3 = 13
(gdb) p $p->tf->a0
$4 = 5
(gdb) p/x $scause
$5 = 0x8
(gdb) p/x $sepc
$6 = 0x17f4
```

The registers firmly show that the syscall number (13) and argument (5) safely crossed the boundary. `scause` is 0x8, which specifically identifies an Environment call from U-mode (a deliberate syscall). The saved program counter, `sepc`, points perfectly to 0x17f4, the exact memory address of the `ecall` instruction that caused the trap.

Inside the kernel, `usertrap()` identifies the syscall and forwards execution to the `syscall()` dispatcher. The trapframe values observed were:

```
(gdb) p $p->tf->a7
$7 = 13
(gdb) p $p->tf->a0
$8 = 5
(gdb) p/x $p->tf->epc
$9 = 0x17f8
(gdb) p/x ($p->tf->epc - $ecallpc)
$10 = 0x4
```

The four-byte difference (0x4) explicitly demonstrates the kernel advancing the saved program counter (`epc`) by the size of the `ecall` instruction itself. This critical step ensures that when the CPU eventually returns to user mode, execution cleanly resumes at the `ret` instruction (0x17f8) rather than infinitely re-triggering the `ecall`.

The dispatcher uses `a7` to map to `sys_sleep()`. The GDB backtrace confirms the deep kernel path:

```
#0  sys_sleep () at kernel/sysproc.c:54
#1  0x000000008020356c in syscall () at kernel/syscall.c:124
#2  0x000000008020275a in usertrap () at kernel/trap.c:134
```

Once the sleep completes, the handler returns success, setting `a0 = 0`. Finally, `usertrapret()` restores the context and switches the CPU back to user mode.

The complete holistic execution path is:

```
sleep(5) → usys.S → ecalls → usertrap() → syscall() → sys_sleep() → usertrapret()
```

Testing summary

Component	Tests performed	Result/evidence
sleep	Tested valid inputs (<code>sleep 0</code> , <code>sleep 5</code>), and heavily tested invalid bounds (<code>sleep -1</code> , mixed text like <code>12x</code> , empty arguments, and extreme overflow values).	All valid and invalid edge cases produced the correct targeted error messages. No system hangs or silent wraparounds were observed. The syscall path was traced perfectly via GDB.
pingpong	Executed the bi-directional pipe communication program repeatedly under both single-core (<code>CPUS=1</code>) and multi-core (<code>CPUS=4</code>) simulated environments.	The program consistently produced exactly two ordered messages (received ping then received pong) with perfect deterministic synchronization across all CPU configurations.
pstree	Tested default root (<code>pstree</code>), explicit root (<code>pstree 1</code>), an absent root, and dynamically tested it while the background <code>vdemo</code> process was rapidly forking and exiting.	It printed the full process hierarchy correctly, with proper parent-child indentation. Siblings were strictly sorted by PID, and the cycle-detection logic safely prevented infinite loops when catching moving PIDs.
syscallmap.py	Verified the generated syscall ABI for consistency. Performed deliberate fault injection testing by modifying the <code>exec</code> stub in <code>usys.S</code> .	The script correctly detected the exact ABI mismatch on <code>exec</code> . When subsequently repaired via <code>gensyscalls.py</code> , it cleanly reported full ABI validity.

Debugging reflection

Bug 1 (sleep.c) - Integer Overflow Handling: While aggressively implementing overflow protection in `sleep.c`, my initial attempt at handling overflow silently wrapped around to a negative number, causing the sleep program to throw an obscure error. When I attempted to redesign the parser, the GCC compiler began throwing an `incompatible pointer type` error. Investigation revealed this was because my `ticks` variable was declared as a `long long` in `main`, but my helper parsing function, `parse_ticks`, was still strictly expecting a standard `int` pointer. I explicitly updated the signature of the `parse_ticks` function to accept a `long long` pointer and applied rigorous boundary checks. This resolved the compilation failure and ensured that my mathematical overflow logic finally worked as intended, successfully catching and rejecting excessively large inputs without wrapping around.

Bug 2 (pstree.c) - Infinite Process Cycles: During the initial implementation of the process tree viewer, the code seemed to work fine on static processes. However, it entered a catastrophic infinite loop when tested against the chaotic `vdemo` process. `vdemo` rapidly forks and exits, immediately

reusing PIDs and creating nasty snapshot inconsistencies where two rapidly-cycling processes technically appeared to be each other's parents in the momentarily captured array. When I ran `pstree`, the terminal hung completely and became unresponsive. By injecting `printf` trace statements, I discovered that my recursive DFS traversal was bouncing infinitely back and forth between two distinct process indices forming an impossible cycle. To fix this, I introduced a boolean `visited` array to rigorously track which absolute array indices had already been printed during the descent. Before recursively descending into a supposed child node, my code now checks if that specific index is already flagged in the array. If it is, it aggressively returns immediately instead of recursing further. This structural patch instantly eliminated the infinite loop and rendered the tree traversal completely immune to cyclic snapshot races.